

Stridelings - Team24

Stridelings - Team24 (*student proposal*)

Assessment: ✓ Fit · **Category:** Games (*gamified-app lens*) · **Assessed:** 2026-09-21 · **Status:** awaiting instructor approval

Proposal (as submitted): *Stridelings* is a **gamified fitness-tracking app** that turns walks and runs into a **creature-collecting game without interrupting the workout**. A tracked session records distance, pace, route and time, with a chosen **Strideling** as companion. The Singapore map is gamified into a **biome grid** (4 biome types), each biome with its own resources and Stridelings of various rarities; resources auto-gather as the user moves and Stridelings follow with interest. At session end, the user keeps the Stridelings that followed to the end and levels their companion by the session; gathered resources decorate a **home base** where creatures hang out. Biomes **rotate weekly**, pushing new routes; daily/weekly missions tailored to the user's goals reward consistency. Named load-bearing C++: a **real-time GPS signal-conditioning pipeline** (Kalman-style filtering/smoothing of noisy GPS + spoofing/abnormality detection); a **real-time 2D map render via OpenGL ES** from **GeoJSON** queried from a **PostgreSQL + PostGIS** server; **offline session recording** (local SQLite) synced to the server at session end for **validation + reward issue** (so recording survives tunnels/deadzones); and a **data-driven content pipeline** (all Stridelings, biomes, resources and drop tables defined externally, edited in the team's editors, exported as **versioned content packs**, shipping content without code changes). The sim loop reads GPS to update distance/pace, determine the current biome, decide resource collection and creature joins. Tools: a **creature-creation tool** (spritesheets, stats, spawn, rarity); a **session simulator** (synthetic GPS traces treated as real sessions with real rewards, for dev testing); a **biome editor** (biome art + collectible resources); and a **debugger/logger** (location/updates, biome changes, creature encounters, rewards, missions, system errors). Design features: an **interaction-free session** (stats screen + map screen with creatures visibly joining, easy switch); the **grid-based rotating biomes**; **creature collecting** (unique companion passives — e.g. more resources / rarer encounters — level-ups unlock passives, duplicates fuse to enhance); and the **home base**

(tap-to-interact creatures + furniture bought with gathered resources). Team: **6 CS + 1 Design + 0 Art**.

Note (category): like other **gamified apps**, Stridelings is filed under **Games** (there is no separate fitness/health category) — assessed through the gamified-app lens (session/economy/retention systems), the same way the module treats step-and-pet and habit-loop projects.

Note (module completeness): the proposal's server is **PostgreSQL + PostGIS** on the team's **Linux server** — exactly the module's Extended tier (a Linux server set up by M2, full use T2). Firebase is the module's default external DB; the team may use it for auth/accounts or keep PostGIS as the primary store (either is fine — just set the seam up + basic-test it by M2). *Monetization:* none is specified and the economy is **earn-by-exercise** (resources → furniture), which is clean; if real-money purchases are ever added, keep them **cosmetic / non-pay-to-win** (no buying rarity or power) per the module's clean-monetization rule — worth stating explicitly given the rarity/fusion systems.

References:

- [Pikmin Bloom — Wikipedia](#) (Niantic's step-tracking companion app: creatures grow from and **follow you as you walk**, collected over real-world movement without interrupting the walk — the fitness-walk + creatures-follow-you precedent)
- [Pokémon Go — Wikipedia](#) (the location-based creature-collecting benchmark: a gamified real-world map with creatures to encounter and collect as the player moves through GPS space — the map-as-game-board + collect-by-moving precedent)
- [Zombies, Run! — Google Play](#) (the benchmark gamified running app that layers a game over the run **without interrupting the workout**, rewarding consistency through missions — the non-interruptive-fitness-game precedent)

Core Tech Requirements

Legend: ✓ Applies directly | ♦ Applies with domain adaptation | ● Optional | ✕ Not needed | » Second trimester

Core Tech Requirement	Stridelings
External Database	✓
Authentication »	●
Analytics »	●
Real-time C++ Simulation Loop and/or C++ Performant Service	✓
Application Communication	✓
Continuous Integration	✓
Data-Driven Architecture	✓
Front-end + Back-end	✓
Custom Tools Development	✓
2D Graphical Output	✓
3D (strong students only)	✕
Networking System	●

How each requirement applies:

Platform fit: **Main:** Native mobile (Native Android — C++ engine via JNI, OpenGL ES map rendered in a GLSurfaceView, Kotlin app shell; device: Pixel 9a) — GPS, background-safe offline recording and on-device SQLite are native concerns. The Native PC build is the development/tools vehicle (the creature-creation tool, biome editor, session simulator and logger). **Extended:** the team's **Linux server + PostgreSQL/PostGIS** (set up by M2, full use T2) serving GeoJSON + validating sessions and issuing rewards; Firebase optionally for auth/accounts.

Front end: A 2D app — the interaction-free session (stats screen + the OpenGL ES map with the biome grid and Stridelings visibly joining), the collection, the companion/level UI and the decoratable home base.

Languages: Native — Kotlin Native Android UI + the C++ engine/services via JNI (Android Studio); OpenGL ES for the realtime map in a GLSurfaceView; some screens (collection, home, missions) can be pure Native UI. Web/desktop — SDL2/3 + OpenGL ES for the PC dev/tools build.

Back end: Two layers. On-device C++: the GPS signal-conditioning pipeline (Kalman filtering + spoof/abnormality checks), the real-time session sim loop (distance/pace, biome resolution, resource collection, creature joins), the map renderer, and the offline SQLite recorder. Server: the Linux + PostgreSQL/PostGIS service that serves GeoJSON, validates uploaded sessions and issues rewards, and hosts the versioned content packs. **Languages:** C++ (device engine/services); server (Postgres/PostGIS + the team's service layer); Firebase optional.

Backend scope: Local + offline now, server-validated (T2) — the upload/validate/reward seam set up & basic-tested by M2, full use in T2.

- *Loop and/or service:* the C++ sim loop reads conditioned GPS each tick to update distance/pace, resolve the current biome cell, decide resource collection and creature joins, and drive the two session screens; the **GPS signal-conditioning pipeline** (Kalman filter + spoof detection) is a distinct C++ performant service feeding it.
- *Data-driven:* Stridelings (stats/rarity/spawn/passives), the 4 biomes + their resource/drop tables, missions and home furniture are all **external content packs** — new content ships without a rebuild (this is a named, load-bearing requirement, not an afterthought).
- *2D:* the OpenGL-ES map render from server GeoJSON, the biome grid, the joining creatures, the collection and the home base (custom-rendered). *3D:* not needed.
- *Front-end + Back-end:* a genuine client/server split — device app + Linux/PostGIS server that validates sessions and issues rewards; the offline-record-then-upload flow is **Application Communication** ✓.
- *Custom tools:* four — the **creature-creation tool**, the **biome editor**, the **session simulator** (synthetic GPS traces), and the **debugger/logger**.
- *Networking:* no real-time multiplayer — sessions upload asynchronously after they end, so a Networking System is optional (●). *External DB:* central

(PostGIS + content packs + per-user collections). *Auth* » / *Analytics* »:
accounts and consistency analytics are natural but module-deferred to T2
(●).

Suggested Tech Goals

G1–G3 ★ are the common goals (fixed for all categories); the other 3 are picked from the Games pool in `milestone-goals-pool-per-category.pdf` (G1–G3 ★ common, G4+ picks).

M1:

- **G1 ★ Application Shell, Real-time C++ Simulation Loop, Input & Core Logic** — the session loop + the **GPS signal-conditioning** service.

Tech & dependencies

- **Tech: Web build:** A fixed-timestep C++ loop reading conditioned GPS to update distance/pace, resolve the biome cell, and decide resource collection + creature joins; a GPS signal-conditioning pipeline (Kalman filter, smoothing, spoof/abnormality checks) feeding it; C++17/20, CMake, Git. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); GPS + OpenGL ES map in a GLSurfaceView; some screens pure Native UI.
- **Prerequisites:** None — foundation of this project.
- **Feeds into:** Every other goal; the map render (M1G4), creature behaviour (M1G8) and the offline recorder (M1G12) run off its state.

- **G2 ★ Data-Driven Architecture of Core Objects / Entity System** — the **content-pack** backbone.

Tech & dependencies

- **Tech:** An entity/data model with Stridelings (stats/rarity/spawn/passives), the 4 biomes + resource/drop tables, missions and furniture in hot-reloadable, ****versioned**** JSON content packs — a new creature or biome is a data change, no rebuild.
- **Prerequisites:** G1 (loop & entity hooks).
- **Feeds into:** M2G2 (creature-creation tool), M2G6 (biome editor + content-pack export), M1G8 (creature data).

- **G3 ★ Debugging, Profiling, Stress Test & CI** — the **debugger/logger** + the **session simulator**.

Tech & dependencies

- **Tech:** A session logger (location, updates, biome changes, creature encounters, rewards, missions, errors), ImGui overlays, a session simulator that feeds synthetic GPS traces through the real loop, Tracy

profiling of a long session, and Catch2 tests (GPS conditioning, biome resolution, reward validation), on a GitHub Actions matrix.

- **Prerequisites:** G1 (loop to instrument).
- **Feeds into:** M2G3 (CI hardening); the session-simulator tool at showcase.

- **G4 2D Graphics** — the real-time **OpenGL ES map render** from server GeoJSON + the joining creatures.

Tech & dependencies

- **Tech: Web build:** An OpenGL ES renderer drawing the map from GeoJSON (polygons/paths), the biome-grid overlay, the user's route, and the Stridelings visibly joining, plus the stats HUD; SDL2/3/OpenGL ES, GLSL, FreeType. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES map in a GLSurfaceView; some screens pure Native UI.
- **Prerequisites:** G1 (loop & window); server GeoJSON.
- **Feeds into:** The session map screen; the biome editor shares this renderer.

- **G8 Enemy AI & Pathfinding** — the **creature spawn + follow behaviour** (Stridelings follow with interest, join/leave).

Tech & dependencies

- **Tech:** Biome-weighted spawning by rarity, steering so creatures follow the moving user with interest, and the join/keep/leave logic (kept if they follow to session end), tuned from creature data (custom C++).
- **Prerequisites:** G1 (session state), G2 (creature/biome data).
- **Feeds into:** M2G13 (collection/level/fuse/passive systems); the collect-by-moving pillar.

- **G12 Save / Persistence & Server Integration** — **offline SQLite recording** + end-of-session upload/validation.

Tech & dependencies

- **Tech:** Local SQLite recording that survives tunnels/deadzones, then an end-of-session upload to the Linux/PostGIS server which validates the trace and issues rewards; REST from C++ (JS SDK / Firebase optional for auth). **Web build** uses the same service API.
- **Prerequisites:** G1 (session data to record), G2 (reward/content data).
- **Feeds into:** M2G8 (server accounts/validation/analytics); reliable rewards.

M2:

- **G1 ★ Architecture — Optimization & Platform Validation** — on-device battery/perf + the Linux/PostGIS server seam.

Tech & dependencies

- **Tech: Web build:** Profiling + optimization of a long session (battery-aware GPS/render cadence), sanitizers, and on-device validation (Pixel 9a); the Linux + PostgreSQL/PostGIS server (GeoJSON + session validation + content packs) set up (account + software + basic connectivity test) by M2. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES map in a GLSurfaceView; some screens pure Native UI.
 - **Prerequisites:** All M1 goals (hardens the M1 codebase).
 - **Feeds into:** The mobile deliverable + a battery-viable long session.
- **G2 ★ Content Editor — Core (Creature-Creation Tool)** — spritesheets, stats, spawn locations, rarity.

Tech & dependencies

- **Tech:** Dear ImGui (or a small external editor): upload a spritesheet, set stats/passives, spawn biomes and rarity, round-trip JSON into a content pack, with live preview and undo/redo.
- **Libraries/Software:** Dear ImGui, nlohmann/json, the project's renderer (preview).
- **Prerequisites:** G2 (creature data model).
- **Feeds into:** All creature content; the custom-tools showcase.

- **G3 ★ CI — CMake, Code Quality, Build Standards & Packaging**

Tech & dependencies

- **Tech: Web build:** CMake presets, clang-format + clang-tidy gates, reproducible Windows + Android builds, tests (GPS conditioning + spoof detection, biome resolution, session validation, content-pack loading — driven by the session simulator's synthetic traces) and packaging via GitHub Actions. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES map in a GLSurfaceView; some screens pure Native UI.
 - **Prerequisites:** G3 (CI & test foundation).
 - **Feeds into:** Release/showcase builds of every goal.
- **G6 Content Editor — Advanced (Biome Editor + Versioned Content Packs)** — biome art/resources + content-pack export.

Tech & dependencies

- **Tech:** Dear ImGui advanced widgets: edit each biome's art assets and collectible resources/drop tables, assemble creatures + biomes + missions into a ****versioned content pack****, and export it so new content ships without rebuilding; live preview binding, undo/redo depth.
- **Prerequisites:** M2G2 (editor core), G2 (data model).
- **Feeds into:** Fast content authoring; T2 live content updates.
- **G8 Server — Accounts, Session Validation, Missions & Analytics** — the PostGIS/Firebase back-end.

Tech & dependencies

- **Tech:** The Linux + PostgreSQL/PostGIS service (or Firebase for auth): per-user accounts + collections, server-side session validation + reward issue, the daily/weekly mission system, the weekly biome-rotation job, and consistency analytics; security so a client can't forge a session or rewards.
- **Prerequisites:** M1G12 (offline record + upload), M2G1 (server set up).
- **Feeds into:** Trustworthy rewards + missions; T2 live-ops.
- **G13 Gameplay Systems II** — the **collection / level / fuse / passives** + the **home base**.

Tech & dependencies

- **Tech:** C++ systems over the entity/data model: the collection, companion level-ups unlocking passives, duplicate ****fusion**** enhancing passives, the resource economy + furniture purchases, and the tap-to-interact home base; balancing-data-driven via the content packs.
- **Prerequisites:** M1G8 (creatures), G2 (creature/resource data).
- **Feeds into:** The meta-loop depth at showcase; T2 content expansion.

Track Goals & Team Composition

Recommended composition: 6 CS + 1 Design + **1 Art** (8) — a creature-collector is an **art-heavy** genre (many Stridelings across rarities, 4 distinct biomes, home-base furniture, UI), so the proposed **6 CS + 1 Design + 0 Art is the module's tight case**: with no artist, the single designer must also carry the **D11 conditional** (Art POC / Art Implementation Quality) each milestone, and a **solo designer + 0 artists** is exactly the situation the design pool flags — such a team "must ask the instructor which conditional bends" (see [design-goals-pool.pdf](#)). **Strongly recommended: add 1 Artist** (best for a monster-collector's asset volume), or at least a **2nd designer** so one owns the session/economy/retention design and the other absorbs the D11 art surface. If the team stays 0-artist, scope the art hard: a **small, reusable, stylised creature/biome set** (a few base silhouettes recoloured by rarity/biome; tile-based biome art) authored/sourced under D11, and expect the instructor sign-off on the bent conditional.

Scope note (advisory): a Kalman GPS pipeline + a GeoJSON OpenGL-ES map + offline-record-and-validate + a data-driven content pipeline + four tools + a client/server split is a **large, server-heavy CS scope** — one biome type, basic GPS→distance/creature-join, offline record + a stub validate, and one editor is the M1/M2 floor; the 4 rotating biomes, spoof detection, fusion/passives, the home base and all four tools are the stretch. The **content-pack pipeline is the smartest lever** here — it lets the (small) team ship creatures/biomes as data without engineering time per item.

Track goals — suggested Design / UI-UX / TEAM picks for this project (no Art track as proposed — 0 artists, so the Art surface is carried by the Design-owned D11 conditional; adding an artist is recommended, see above); deliverables & quality ladders live in the track pool documents.

- **Design M1:**

- M1D1 ★ Features Design Document (the gamified-fitness USP, the biome grid + weekly rotation, the creature-collect/level/fuse economy, the home base and the non-interruptive session; primary benchmark e.g. Pikmin Bloom / Pokémon GO, plus a Singapore benchmark — Healthy 365 fits the local-awareness criterion)
- M1D9 Engagement & Retention Loop Design (the daily/weekly missions tied to the user's goals, the biome-rotation variety loop)

and the collection meta — with the ethics line: engagement serves the user's fitness goal, never dark-pattern or pay-to-win)

- M1D11 Art Proof of Concept *(conditional — mandatory for 0-artist teams; Design-owned)* — the stylised creature/biome visual direction: a small reusable creature set recoloured by rarity/biome, the 4 biome looks, home-base style; 2 mockups.

- **Design M2:**

- M2D1 ★ Features Design Iteration & Showcase Readiness
- M2D11 Art Implementation Quality *(conditional — mandatory for 0-artist teams; Design-owned)* — the creature/biome/home art implemented in-engine, no placeholders.

- **UI/UX M1** (graded by the Design instructor for the Games category):

- M1U1 ★ UI Standards & Wireframe Baseline — readability standards for the two session screens (glanceable stats + the map with joining creatures), the collection and the home base, plus a wireframe baseline; special attention to **glanceability while moving** (the non-interruptive session).

- **UI/UX M2:**

- M2U1 ★ Functional UI & Usability — the session screens, collection, companion/level, missions and home base navigable unaided on the Native-Android target.
- M2U2 ★ UX Validation & Final Polished UI — validated, polished non-interruptive-session + collection UX tested with real walkers/runners on the Pixel 9a.

- **TEAM M1:**

- M1TA1 ★ Overall Audio Experience — initial BGM/SFX from the week-5 needs list present & working in the prototype (ambient session BGM, resource-collect, creature-join, level-up, mission-complete cues), suitable and correctly triggered.
- M1TA2 ★ Audio Engine Functionality — data-driven audio pipeline (add/replace files without recompiling) + reliable multi-channel playback.

- **TEAM M2:**

- M2TA1 ★ Overall Audio Experience — standard floor (ambient BGM + collect / creature-join / level / fuse / mission / home SFX, ≥6 types), suited to a non-interruptive, often screen-off session.

- M2TA2 ★ Audio Engine Functionality — audio properties de/serialised and the full playback functionality the build needs (multi-channel mixing; distinct creature-join + reward cues).
- M2TP1 ★ Final Presentation — 7 min in front of class + instructors: live custom-tools showcase (the creature-creation tool + the biome editor + the session simulator), prototype demo, team post-mortem (wk14)

*Generated by the Project Proposal Assessor skill · grounding: live folder
(Project 2 Technical adjustments/) · references verified 2026-09-21.*